

All the I76/Nitro On Disk Formats (That I Know About)

Note: My original blog is at: [HackingOnSpace](http://hackingonspace.blogspot.co.uk/) (<http://hackingonspace.blogspot.co.uk/>). Go there for more details on the reverse engineering process and examples of code, scripts and test apps.

Introduction

This document is the result of reverse engineering the on disk data files for game content. It's been generated by examining the installed game files and public information on similar games of the era from the same publisher. No proprietary Activision information was used to generate this document.

Suggestions, edits and corrections are welcome. Post changes up at <http://forums.interstate76.com/> and flag for the user *tonyPick*.

Big Thanks go out to

Lightfoot & D1VER; from the [I76 forum](#). Also check out [D1VER's Homepage](#) for much stuff.

Erik; a blog commenter who has some of the assets up in unity engine.

Karl Meissner & Kurt Arnlund; of the original I76 development team.

Hoppo; for an early I76 hacking site that saved me months of trial and error.

Many other internet sites, some only available via the wayback machine, for decoding various pieces of the puzzle. See the blog.

If you are at all interested in this game then go over to <http://forums.interstate76.com/> for live multiplayer servers, patched binaries and extra maps. At the time of writing you can get both I76 and Nitro Riders from GoG at <https://www.gog.com/>. And you should. They are still awesome. And *Funky*, dig?.

Overview

This document breaks the files down into one of two types:

- Fixed Format files: These have a prescribed format, such as the *ZFS* or *geo* files
- Flexible Format Files: These have a generic “header/length/data” structure, and the content is determined by the order of tags in any specific file. Examples are the *WDF*, *MSN* or *VDF*

I call the flexible format files “BWD2” format, based on the opening tag.

Data Types

The file format data is Intel Little Endian; lowest byte first. The following primitives are used in the format breakdown:

char

Single Byte Character. Encoding is ASCII throughout, so for example 0x49 (79 decimal) is ‘O’.

byte

Also a single byte Character: Char and Byte are used interchangeably in this document, although bytes tend to reference raw data, and char tends to reference ASCII characters in strings or tags.

uint16_t

Unsigned 16 bit integer. Two bytes of file data.

examples

So if file hex data is : “0x10 0x2D”

Then the 16 bit value is 0x2D10

Or 11536 decimal

uint32_t

Unsigned 32 bit integer. Four bytes of file data.

examples

If the file hex data is “0x11 0x22 0x33 0x44”

Then the 32 bit value is 0x44332211

or 1144201745 decimal

If the file hex data is “0xb0 0x1a 0x00 0x00”

Then the 32 bit value is “0x00001ab0”

or 6832 decimal

float_t

32 bit floating point value, held as IEEE-754 Single Precision floating point value (binary32). Four bytes of file data.

This means the floating point number is

1 bit sign

8 bit exponent

23 bit fraction

Floating point numbers are fiddly to do manually. The good news is that many Hex Editors will interpret this type for you, and if coding on a PC then the conversion is a simple cast operation from the raw bytes to the C *float* type.

For the full information see wikipedia

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

For an online conversion routine that breaks down the details:

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

examples

If the file data is “0x00 0xd0 0xd9 0x44”

Then the 32 bit value is “0x44d9d000”

Which is sign: 0, exponent 137, fraction 5885952

Or Exponent 10, Mantissa 1.70166015625

or 1742.5 decimal floating point

If the file hex data is “0x80 0xe3 0x43 0x47”

Then the 32 bit value is “0x4743e380”

Which is sign: 0, exponent 142, fraction 4449152

Or Exponent 15, Mantissa 1.5303802490234375

or 50147.5 decimal floating point

Arrays: []

Square brackets indicate arrays, using the standard C notation.

So, for example, “*uint32_t*[4]” indicates a set of four *uint32_t* values

The Fixed Format File Types

ZFS

The main compressed archive. This consists of

Section	Size	Data
Archive Header	28 Bytes	archive_hdr_t
'n' Directory Headers, as determined by the archive header	n * 36 Bytes	directory_hdr_t[n]
File Data	Remaining File Size	Raw data bytes

Where

archive_hdr_t

char[4] ident_string	Always ZFSF
uint32_t version	Always 1
uint32_t unknown	Always 0x10 (more version?)
uint32_t files_per_directory	The number of files each directory entry will describe
uint32_t total_files	The total number of files in this archive
uint32_t null_marker	Always 0
uint32_t unknown	

directory_hdr_t

uint32_t next_directory	Offset to the next directory header
<i>Then for each file in directory</i>	
As determined by files_per_directory	
char[16] name	Name of the file
uint32_t data_offset	Position of file data in this archive
uint32_t id	File id
uint32_t data_length	Length of Data in the archive
uint32_t unknown	
byte compress	Compression Flags (<i>always 0 for I76, sometimes set in Nitro</i>) 0 = no compression 2 = LZO 1x 4 = LZO 1y
uint24_t output_size	Decompressed file size (0 if no compression)

MW2

Another compressed archive, but with fewer (less interesting) files and no name/directory structures.

Section	Size	Data
Archive Header	Variable	mw2_hdr_t

Where

mw2_hdr_t

uint32_t file_count	Number of files
<i>For each file</i>	As determined by file_count
uint32_t data_offset	Position of file data in this archive

Geo

Geometry file. Describes a set of vertices and faces, which represent a model.

Section	Size	Data
Geometry Header	36 bytes	geo_hdr_t
‘n’ Vertex Positions	n * 12	vertex_t
‘n’ Vertex Normals	n * 12	normal_t
‘m’ Faces	Variable (determined by vertex per face)	face_t

geo_hdr_t

char[4] ident_string	Always “EOG.”
uint32_t unknown	
char[16] name	Associated name to this object geometry
uint32_t vertex count	
uint32_t face_count	
uint32_t unknown	

vertex_t

float32_t[3] vertex_data	X, Y & Z values
--------------------------	-----------------

normal_t

float32_t[3] normal_data	X, Y & Z values
--------------------------	-----------------

face_t

uint32_t face_index	
uint32_t face_vertices	
char face_r	
char face_g	
char face_b	

float32_t[4] surface_normal	
uint32_t <i>unknown</i>	
char[3] <i>unknown</i>	Face flags?
char[13] texture_name	
uint32_t <i>unknown</i>	
uint32_t <i>unknown</i>	
<i>For each face vertex</i>	As given by face_vertices
uint32_t vertex_index	
uint32_t normal_index	
float32_t U	
float32_t V	

Pix & Pak

These files are more packed data files. The Pix file supplies the index, and the Pak file is the raw data

The index file, pix, is a plain text file, which contains

- *a count of entries*

Then for each entry

- *<file name> <start in Pak> <length in Pak>*

WAV

Sound clips. These are plain .wav files. You can play them in a stock media player.

CBK

Colour Bank file. This describes a set of 4x4 pixel block pattern used to generate textures.

uint32_t entries	Count of entries in this file
<i>Then, for each entry</i>	
char[16] pattern	4x4 pattern, each byte references a LUT entry

VQM

A texture image. This references the 4x4 pixel patterns in a CBK file to generate the texture. The texture width and height do not have to be divisible by 4, and code must handle this and ignore pixels that fall outside the image bounds.

uint32_t pattern_width	
uint32_t pattern_height	
char[16] cbk_ref	Colour bank file associated with this pattern

<i>Then for each 4x4 block</i>		Describe each 4x4 pixel block. Either a solid colour or pattern reference
	uint16_t reference	Pattern reference if “reference & 0x8000” Fill 4x4 block with the solid colour from the CLUT entry given by reference &0xff else Fill 4x4 block with the CBK pattern at this index

MAP

Simple bitmap files, with a single byte per pixel taken from the CLUT.

unit32_t bitmap_width		
unit32_t bitmap_height		
<i>Then for each pixel</i>		
	byte clut_ref	Pixel Colour value from CLUT

TER

Terrain file. Each terrain file contains one or more 32K blocks, each of which supplies a 128x128 block of terrain height values, as a 16 bit heightmap. Only the lower 12 bits of each value are used for height, and the higher bits are used to flag terrain properties.

Each bit of the heightmap covers a 5 Meter square, and each terrain block of 128*128 values is a 640 x 640 meter region.

Section	Size	Data
‘n’ Terrain blocks	n * 32KBytes	terrain_block_t

Where:

terrain_block_t

uint16_t terrain[128][128]	Terrain Value terrain & 0xffff gives the 12 bit height value terrain &0xf000 gives terrain detail flags.
----------------------------	--

BWD2 Files

Overview

The BWD2 files are used to support several different types of game file.

BWD2 files have a flexible format. Each file is made up of several data sections, each of which has the following layout:

char[4] identifier	Type of section
unit32_t length	Length (including the identifier)

byte[length-8] data	Associated data, format details determined by identifier
---------------------	--

BWD2 files are used for files of type msn, lvl, cbt, rac, vdf, gdf, wdf, cdf,vcf,sdf,vtf & xdf.
Possibly others.

A BWD2 File Opens with

Tag: BWD2

Length: 8

Data: No data

And Ends With

Tag: EXIT

Length: 8

Data: No data

(For both of those – the length is “4 byte tag, 4 byte length and nothing else”)

The data can contain nested tags, (i.e. the data payload of a section contains a number of other bwd2 tag/length/data sets) and there can be several EXIT tags in the file, since some sections also place a final EXIT tag.

Generally a given type of BWD2 file will tend to have the same set of content tags in the same order. As an example a mission file (.msn, .cbt or .rac) will have the following tags:

- BWD2
- REV
- WDEF
- TDEF
- RDEF
- ODEF
- LDEF
- ADEF
- EXIT

In this case that's:

- Initial BWD2: Tells us it's a BWD2 file
- A Version tag.
- World information (Level name, intro & outro movies, soundtrack, time of day, etc)
- Terrain Map (The height-field that gives you the terrain)
- Road Definitions (Where the roads get drawn on the terrain)
- Object Definitions (Things like buildings, spawn points and bridges)
- String Object Definitions (Things like fences, where one object repeats a number of times)
- Scripts (The high level logic used to run the game: stuff like “go here” and “destroy that”)
- Final EXIT

Some tags may be 0 sized (such as the ADEF section in multiplayer maps, where there is no scripting).

The Tags

REV

uint32_t revision	Main File Revision
-------------------	--------------------

WDEF

World definition from mission files

WREV	BWD2 Revision Info
WRLD	BWD2 World Info
EXIT	

WREV

uint32_t revision	World Data revision. Modifies WRLD content Revision 8 is I76 Revision 9 is Nitro
-------------------	--

WRLD

uint32_t cd_track	
char[13] intro_movie	
char[13] outro_movie	
char[13] pallette_file	
char[13] luma_file	
char[13] trans_file	
char[13] objectives_file	
char[13] sky_file	
char[13] scrounge_file	
char[13] surface_file	
char[13] level_file	
<i>if version == 9</i> char[13] hzd_file	Only present in Nitro
uint32_t time	
float32[40] <i>unknown</i>	
uint32_t clip_distance	
char[16] level_name	

TDEF

Terrain Definition

TREV	Revision Info
ZMAP	Zone Map
ZONE	Zone Info
EXIT	

TREV

uint32_t revision	Terrain Data revision. Always 4
-------------------	---------------------------------

ZMAP

Zone Map. This describes an area of 80x80 terrain zones. Each zone is a single 128*128 TER block reference from the *.ter* file (named in the ZONE section).

byte max_zone	count of zones active on this map
byte [6400]	80x80 map of zone references if value == 0xff then default flat zone else reference to .ter file zone

ZONE

Zone information

byte <i>unknown</i>	
char[13] zonemap	Name of the terrain file providing the zones indexed by the ZMAP

ODEF

“simple” single objects

OREV	BWD2 Revision Info
OBJ[]	BWD2 Objects
EXIT	

OREV

uint32_t revision	Object Data Revision (Always 3?)
-------------------	----------------------------------

OBJ

Object (single version)

char[9] label	
float32_t[9] rotation	

float32_t[3] position	
uint32_t[9] <i>unknown</i>	
uint32_t class_id	
uint32_t <i>unknown</i>	

LDEF

“linked” object (or stringed object, in map editor speak)

LREV	BWD2 Revision Info
OBJ[]	BWD2 Objects (linked version)
EXIT	

LREV

uint32_t revision	Object Data Revision
-------------------	----------------------

OBJ

Object (Linked Version)

char[9] label	
float32_t[9] rotation	
float32_t[3] position	
uint32_t[9] <i>unknown</i>	
uint32_t class_id	
uint32_t <i>unknown</i>	
uint32_t string_count	
<i>Then, for each string value</i>	As given by string_count
float32_t[3] point	A co-ordinate for a point in the string

RDEF

Road Definition

RREV	BWD2 Revision Info
RSEG[]	BWD2 Road Segment
EXIT	
EXIT	
EXIT	
EXIT	

For some reason RDEF sections have extra EXIT declarations trailing (consistently four?). I have no idea why.

RREV

uint32_t revision	Road Data Revision
-------------------	--------------------

RSEG

Object

uint32_t type	Road Surface Type
uint32_t segment_count	
<i>Then for each segment</i>	As given by count
float32_t[3] start	
float32_t[3] end	

VDFC

Vehicle Descriptor. This looks to be the central definition of each vehicle.

char[20] name	Name of the Vehicle
uint32_t <i>unknown</i>	
uint32_t <i>unknown</i>	
float32_t[4] <i>unknown</i>	
uint32_t <i>unknown</i>	
float32_t[4] <i>unknown</i>	

VSHL

Links a bmp to a vehicle

char[45] bmp_name	Name of the Vehicle BMP
-------------------	-------------------------

VGEO

Vehicle Geometry Parts. The component pieces of model geometry that are combined to make a single in game vehicle, and some of the internal geometry (for the in car views when using the handgun).

uint32_t unknown	
<i>Then for each segment</i>	As given by data size,100 bytes per entry. The grouping looks to be 31 entries per model.
char[8] Label	
float32_t[12] fv	0-8: 3x3 transform matrix (probably) 9:X 10:Z 11:Y.
char[8]	The part this position is relative to. This is either a part label or

	position_root	WORLD for global positions
	byte[36] <i>unknown</i>	

XDFC

uint32_t <i>unknown</i>	
float32_t <i>unknown</i>	
float32_t <i>unknown</i>	
byte[12] <i>unknown</i>	
float32_t <i>unknown</i>	
float32_t <i>unknown</i>	
float32_t <i>unknown</i>	
uint32_t <i>unknown</i>	

SOBJ

A VGEO style geometry fragment. Giving a label and transform information.

char[8] Label	
float32_t fv[12]	
char[8] position_root	
byte[36] <i>unknown</i>	

ETBL

char[13] ident string	Map name
uint32_t <i>unknown</i>	
uint32_t section_count	
byte <i>unknown</i>	
Then for each segment	As given by section_count
	char[16] label
	uint32_t <i>unknown</i>
	uint32_t <i>unknown</i>
	uint32_t <i>unknown</i>
	uint32_t <i>unknown</i>

VCHK

Part of the vehicle definition

char[8] label	
uint32_t section_count	

byte unknown		
<i>Then for each segment</i>		As given by section_count
	char[8] label	
	float32_t[12] fv	
	char[8] root	(Always WORLD?)
	byte[36] unknown	

ADEF

This is the mission script and associated references (audio, triggers, etc).

This is a fairly complex section. The scripting is run on a small stack based virtual machine.

AREV	BWD2 Revision Info
FSM	BWD2 Finite State Machine
EXIT	

AREV

uint32_t revision	Object Data Revision
-------------------	----------------------

FSM

The FSM section describes a set of stack based virtual machines that run during single player missions to process the higher level game logic, such as having the player follow vehicles, reach waypoints or destroy targets. These machines were compiled from game scripts into microcode using Activision internal tools, and the results are stored in this section.

The FSM section describes several independent machines, and every game engine “tick” then each machine is run in turn. All machines in a given file share a common microcode area. For complex actions the machines have a special instruction (ACTION) which is used to issue commands to, and receive results from, the main game engine.

The main FSM structure is:

uint32_t action_count		
<i>Then for each action</i>		As given by action_count
	char[40] name	These are action aliases, and this table is always the same
uint32_t entity_count		
<i>Then for each entity</i>		

	char[40] aliasname	
	char[8] itemname	
uint32_t audio_count		
<i>Then for each audio clip</i>		
	char[40] clipfile	
uint32_t path_count		
<i>Then for each path</i>		
	char[40] pathname	
	uint32_t point_count	
	<i>Then for each point</i>	
	float32_t[3] pt	
uint32_t machine_count		
<i>Then for each machine</i>		
	uint32_t[42] machine_def	
uint32_t argument_count		
<i>Then for each argument</i>		
	uint32_t arg	A set of initial arguments, which are referenced by individual machines
uint32_t microcode_count		Count of operation/argument pairs
<i>Then for each microcode</i>		
	uint32_t op	opcode
	uint32_t val	Argument value

Machine Definition

Each machine represents a compiled game script, and is defined as follows:

uint32_t start_address	Initial start address in microcode
uint32 arg_stack_sz	
uint32[40] arguments	arg_stack_sz arguments used 0's for unused

Each running machine tracks:

- The Last Action Result (AR)
- The Base of the loaded microcode (IB)
- The Current Instruction Pointer (IP)
- The Base of the Stack (SB)
- The Current Stack Pointer (SP)
- An Argument Stack for passing to Actions (AS)

The game keeps a list of machine details loaded from the mission file, and loops through this list, with each machine being run in turn. A machine relinquishes control using a suitable op-code, but the engine also has a hardcoded upper limit on the number of instruction loops a machine can perform without being de-scheduled.

OpCode Summaries

1	PUSH	Stack Push	Argument written at SP Increment SP
2	Unused		
3	Unused		
4	COPY_S	Copy Argument From Stack.	Copy from "BP+argument" to AS
5	COPY_B	Copy Argument from Parameter.	Copy from "BP + argument-1" to AS. Negative Arguments Expected.
6	STACK_MOD	Add Elements to stack.	$SP = SP + \text{argument}$.
7	POP	POP Elements from stack.	$SP = SP - \text{argument}$
8	JMP	Unconditional Jump.	Set IP to argument
9	JZ	Conditional Jump.	If AR is 0 then set IP to argument
10	JMP_I	Unconditional Jump and De-schedule.	Set IP to argument Suspend the running instance
11	Unused		
12	RST	Reset Machine to base state	Reload IP Reset SP
13	ACTION	Perform Action given by argument.	Consumes AS Issues action to main game engine Modifies AR with result
14	NEG	Negate Result	Complement AR
15	Unused		

The CLUT

This is the RGB colour Lookup Table used by all the game textures. Erik thinks that later entries are transparent, based on surface flag. I can neither confirm, nor deny.

```
static unsigned int LUT[256] = {
    0x000000,
    0xf0e0b0, 0xf0e898, 0xf0d090, 0xe0c088, 0xd0b090, 0xd09898, 0xd0b8b0, 0xb8b8b0,
    0xa0b0c8, 0xb0c0d8,
    0xc0d8e8, 0x78b0d0, 0x8898a8, 0x7888a0, 0x688090, 0x687880, 0x607080, 0x586878,
    0x506070, 0x505868,
    0x485068, 0x405870, 0x305878, 0x386888, 0x507088, 0x4888a8, 0x2870a0, 0x0870c0,
    0x005fb3, 0x005898,
    0x004888, 0x003878, 0x003060, 0x082858, 0x104060, 0x185060, 0x304860, 0xb56f4b,
    0x304058, 0x303850,
    0x203850, 0x283860, 0x404040, 0x503838, 0x505038, 0x486038, 0x606838, 0x587020,
    0x495e1c, 0x405020,
    0x403820, 0x482818, 0x401810, 0x400800, 0x2e0600, 0x1f0400, 0x874827, 0x101820,
    0x202020, 0x282820,
    0x302820, 0x202838, 0x102838, 0x081038, 0x000058, 0x080878, 0x000890, 0x9c5936,
    0x3018b0, 0x5810a0,
    0x900868, 0xa03050, 0x904850, 0x805850, 0x886860, 0x907868, 0x807870, 0x808868,
    0x788858, 0x432828,
    0x3b2020, 0x805830, 0xa06030, 0xa86010, 0x985808, 0x805008, 0x883010, 0x783018,
    0x74280c, 0x602810,
    0x584020, 0x883030, 0x981028, 0x801020, 0x800010, 0x780008, 0x680008, 0x461008,
    0x900010, 0xa80808,
    0xc00818, 0xe00810, 0xf81010, 0xe03800, 0xe84800, 0xf84000, 0xf85008, 0xf85008,
    0xe07008, 0xde7003,
    0xe67800, 0xd88000, 0xd89800, 0xe89800, 0xf0a000, 0xffb200, 0xffc100, 0xffcc04,
    0xfdaf1a, 0xf0a028,
    0xe09020, 0xf07030, 0xfa7338, 0xf0a048, 0xe8a848, 0xd0a048, 0x6d371b, 0xa89060,
    0xa89048, 0xa89828,
    0x809828, 0x708828, 0x588820, 0x619b1e, 0x6db21c, 0x79cc1b, 0xd0c828, 0xe8e830,
    0xf8e858, 0xf8d060,
    0xfcc56d, 0xf0c058, 0xe07868, 0xd86060, 0xe05860, 0xc54969, 0xb05060, 0xb86870,
    0xc07070, 0x989880,
    0xa09890, 0xc86030, 0xc06008, 0xc85808, 0xcb5c00, 0xd06800, 0xc03808, 0xa83010,
    0xa03008, 0xe83058,
    0x389058, 0x188820, 0x088913, 0x086810, 0x015801, 0x004000, 0x003400, 0x012801,
    0x260c00, 0x084820,
    0x105030, 0x007830, 0x1088e0, 0x38a0d8, 0x1010e8, 0xb79c70, 0xb39668, 0xaf905f,
    0xa98954, 0x9f793f,
    0x93713a, 0x8e6d38, 0x886936, 0x826433, 0x7c5f30, 0x75592e, 0x6d532b, 0x654d28,
    0x5b4524, 0x4f3c1f,
    0x372a16, 0xf7f7c8, 0xfae68c, 0xfcdc61, 0xf8c355, 0xdac078, 0xdfa66c, 0xe6a336,
    0xf2884e, 0xe3881c,
    0xb08b44, 0xfc6330, 0xde6b18, 0x9e610d, 0xab4400, 0x732828, 0x930800, 0x414820,
    0x282814, 0x004500,
    0x387338, 0x3f8f3f, 0x519c51, 0x63bd63, 0x7ef37e, 0x502f1b, 0x790018, 0xa10020,
    0xd10028, 0xfa0032,
    0x9e9a1a, 0xccc722, 0xee602, 0xfcfcfc, 0xf3f8fb, 0xedf3fa, 0xe5f0f9, 0xddecf7,
    0xd5e8f6, 0xcee3f5,
    0xc6e0f3, 0xbdcdf3, 0xb6d9f2, 0xafd5f1, 0xa7d1f0, 0x9fcdee, 0x97c9ed, 0x90c5ec,
    0x88c1eb, 0xffffffff,
    0xe00000, 0xddddddd, 0xcccccc, 0xbbbbbbb, 0xaaaaaaa, 0x999999, 0x888888, 0x767676,
    0x656565, 0x545454,
    0x434343, 0x323232, 0x202020, 0x101010, 0x010101,
};
```

Random Other Things

This document is licensed Creative Commons Attribution-ShareAlike 4.0 International License. Basically credit the original, share under the same terms. See <https://creativecommons.org/licenses/by-sa/4.0/>

All trademarks, characters, names, blah blah blah – As far as I know they're all Activision's.